Task 4: Improve GitHub Actions CI Pipeline

Objective

The objective of this task was to analyze and improve the existing GitHub Actions CI pipeline for the DevOps CRM project. The improved pipeline automates security checks, code quality checks, testing, integration testing, application building, and build output validation.

Workflow Created

A new workflow was created at:

.github/workflows/ci-improvements.yml

The workflow is triggered for pull requests when they are opened, synchronized (updated), or reopened.

CI Workflow Steps

1. Checkout Repository

The repository source code is checked out using actions/checkout@v4.

2. Enable Corepack

Corepack is enabled so that the project package manager configuration is available.

3. Setup Node.js

Node.js is configured using the version defined in the project's .nvmrc file, with Yarn caching enabled.

4. Verify Environment

The workflow displays the installed Node.js and Yarn versions to verify the CI environment.

5. Install Dependencies

Dependencies are installed using yarn install --immutable to ensure they match the existing lock file.

6. Security Audit

yarn npm audit --all is executed to check dependencies for known security vulnerabilities.

7. Lint

yarn lint is executed to identify code quality and style issues.

8. Type Check

yarn typecheck is executed to detect TypeScript type-related errors.

9. Unit Tests

yarn test:unit is executed to validate individual parts of the application.

10. Start Twenty Test Server

A Twenty development test instance is started using the Twenty reusable GitHub Action.

11. Wait for Server Health

The workflow checks the /healthz endpoint and retries before failing if the server is not ready.

12. Run Integration Tests

Integration tests are executed with the TWENTY_API_URL and TWENTY_API_KEY provided by the test server action.

13. Build Application

yarn twenty dev:build generates the application manifest and build files while also performing type checking.

14. Validate Build Output

The workflow verifies that important generated files and directories exist in .twenty/output.

Additional CI Improvements

Minimal Permissions

The workflow uses contents: read permissions only. This follows the principle of least privilege by granting only the permissions required by the workflow.

Concurrency Control

Concurrency control was added so that older workflow runs can be cancelled when new commits are pushed to the same pull request. This avoids unnecessary duplicate runs.

Workflow Timeout

A timeout of 20 minutes was configured to prevent the workflow from running indefinitely.

Issue Encountered During Integration Testing

During the first CI execution, the integration tests failed because a page layout tab used the VERTICAL_LIST layout mode while its widget configuration contained a gridPosition, which corresponds to a grid-based layout.

> ***INVALID_PAGE_LAYOUT_WIDGET_DATA: Position layoutMode "GRID" does not match tab layoutMode "VERTICAL_LIST"***

Root Cause

The file src/page-layouts/main-page.page-layout.ts contained a gridPosition configuration for a widget inside a VERTICAL_LIST tab.

Solution

The incompatible gridPosition configuration was removed from src/page-layouts/main-page.page-layout.ts. The application build was then verified locally, and the corrected changes were pushed to the pull request.

Final CI Pipeline

1. Checkout repository
2. Enable Corepack
3. Setup Node.js
4. Verify environment
5. Install dependencies
6. Run security audit
7. Run linting
8. Run type checking
9. Run unit tests
10. Start Twenty test server
11. Wait for server health check
12. Run integration tests
13. Build the application
14. Validate build output

Result

The improved GitHub Actions CI workflow completed successfully after the page layout configuration issue was resolved. The final pipeline provides automated security auditing, code quality checks, testing, integration testing, application build validation, minimal permissions, concurrency control, and timeout protection.

Conclusion

This task improved the reliability and coverage of the project's pull request validation process. The CI pipeline now automatically checks important parts of the application before changes are merged.